

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và xây dựng hệ thống thi trắc nghiệm
online - JQK Study**

TRẦN ĐỨC AN

an.td224912@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: ThS. Lê Đức Trung

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và xây dựng hệ thống thi trắc nghiệm
online - JQK Study**

TRẦN ĐỨC AN

an.td224912@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: ThS. Lê Đức Trung

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy cô giáo hướng dẫn, người đã luôn dành thời gian quý báu để định hướng, tận tình chỉ bảo và hỗ trợ em trong suốt quá trình nghiên cứu và thực hiện đồ án tốt nghiệp này. Sự động viên và những góp ý chuyên môn của thầy cô là kim chỉ nam giúp em hoàn thiện đề tài một cách tốt nhất.

Em cũng xin gửi lời tri ân tới toàn thể các thầy cô giáo trong Khoa Công nghệ thông tin đã truyền đạt cho em những kiến thức nền tảng quý báu và tạo điều kiện học tập tốt nhất trong những năm học qua.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc tới gia đình, bạn bè đã luôn bên cạnh chia sẻ khó khăn, cổ vũ tinh thần và tạo động lực to lớn để em hoàn thành chặng đường đại học và đồ án tốt nghiệp này. Do kiến thức và thời gian có hạn, đồ án không tránh khỏi những thiếu sót, em rất mong nhận được những ý kiến đóng góp của các thầy cô giáo để đề tài được hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án tốt nghiệp này tập trung vào việc nghiên cứu, thiết kế và xây dựng **JQK Study** - một hệ thống học tập và thi trắc nghiệm trực tuyến toàn diện, áp dụng kiến trúc Microservices hiện đại. Trong bối cảnh chuyển đổi số giáo dục ngày nay, các hình thức kiểm tra trực tuyến ngày càng phổ biến nhưng vẫn gặp nhiều hạn chế lớn như: khó kiểm soát gian lận khi học viên làm bài từ xa, quy trình sinh đề thi thủ công mất thời gian, và sự phân mảnh giữa các hệ thống quản lý học tập (LMS) với các công cụ tổ chức thi độc lập.

Để giải quyết các thách thức trên, đề tài đề xuất giải pháp tích hợp đồng bộ hai quy trình học tập và kiểm tra trên cùng một nền tảng JQK Study. Hệ thống được triển khai dựa trên kiến trúc Microservices sử dụng ngôn ngữ **Go** hiệu năng cao cho phần Backend, kết hợp với framework **Next.js** và **TailwindCSS** cho phần Frontend. Điểm nổi bật của giải pháp là cơ chế giám sát chống gian lận (Anti-Cheat) đa lớp trong thời gian thực: ở phía Client-side, hệ thống bắt các sự kiện chuyển tab, blur màn hình hay sao chép dữ liệu thông qua Page Visibility API và Clipboard API để ghi nhận nhật ký vi phạm; ở phía Server-side, hệ thống duy trì bộ đếm ngược thời gian độc lập để tránh can thiệp cục bộ từ trình duyệt. Bên cạnh đó, hệ thống tích hợp thuật toán sinh đề tự động ngẫu nhiên theo cấu hình tỷ lệ độ khó và chủ đề do giảng viên thiết lập sẵn.

Kết quả đạt được của đồ án là một phần mềm hoàn chỉnh chạy trên môi trường container hóa (Docker và Kubernetes) với các dịch vụ lõi bao gồm API Gateway, User Service, Course Service, Exam Service và Notification Service. Hệ thống không chỉ giúp giảng viên tiết kiệm phần lớn thời gian biên soạn đề thi, tổ chức đánh giá lớp học mà còn cung cấp cơ sở dữ liệu giám sát trực quan, đảm bảo tính công bằng và nghiêm túc cho kỳ thi.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

This graduation thesis focuses on researching, designing, and building **JQK Study** - a comprehensive online learning and multiple-choice testing platform based on a modern Microservices architecture. In the context of digital transformation in education, online evaluation has become increasingly popular, but existing platforms face significant challenges: difficulty in monitoring student cheating during remote exams, time-consuming manual exam generation, and the fragmentation between learning management systems (LMS) and independent testing tools.

To address these challenges, this project proposes an integrated solution that unifies learning and testing workflows into a single platform. The system is implemented using a Microservices architecture, featuring high-performance **Go** for the backend services and **Next.js** with **TailwindCSS** for the frontend interface. The highlights of the proposed solution include a real-time, multi-layered anti-cheat monitoring mechanism. At the client-side, the platform captures events such as tab switching, window blurring, and copy/paste actions utilizing Page Visibility and Clipboard APIs to log violation attempts. At the server-side, the system maintains an independent countdown timer to prevent local browser tampering. Additionally, the system features an automated exam generation algorithm that randomly selects questions based on difficulty levels and topic structures defined by the instructor.

The final outcome of this thesis is a fully functional platform deployed on a containerized environment (Docker and Kubernetes) with core services including API Gateway, User Service, Course Service, Exam Service, and Notification Service. The system helps instructors save significant time in preparing exams, manage classroom progress, and provides visualized violation logs, ensuring fairness and integrity in online assessments.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	3
2.1 Khảo sát hiện trạng và các giải pháp liên quan.....	3
2.1.1 Khảo sát các hệ thống giáo dục và thi trực tuyến hiện hành	3
2.1.2 Bảng so sánh chi tiết các hệ thống	4
2.1.3 Nhu cầu thực tế và định hướng giải quyết của JQK Study	5
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã Quản lý người dùng	7
2.2.3 Biểu đồ use case phân rã Quản lý ngân hàng câu hỏi và đề thi	8
2.2.4 Biểu đồ use case phân rã Làm bài thi	8
2.2.5 Quy trình nghiệp vụ	9
2.3 Đặc tả chức năng	10
2.3.1 Đặc tả Use case Sinh đề thi tự động	10
2.3.2 Đặc tả Use case Giám sát và Nộp bài thi	10
2.4 Yêu cầu phi chức năng	10
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	12
3.1 Kiến trúc Microservices	12
3.2 Ngôn ngữ lập trình Go (Golang)	12
3.3 Framework Next.js.....	13

3.4 Giao thức truyền thông gRPC và Protocol Buffers	13
3.5 Hệ quản trị cơ sở dữ liệu PostgreSQL	14
3.6 Hệ thống lưu trữ đệm Redis	14
3.7 Hệ thống hàng đợi thông điệp Apache Kafka.....	14
3.8 Công nghệ Container hóa và Điều phối (Docker & Kubernetes)	14
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	16
4.1 Thiết kế kiến trúc.....	16
4.1.1 Lựa chọn kiến trúc phần mềm	16
4.1.2 Thiết kế tổng quan.....	16
4.1.3 Thiết kế chi tiết gói (Package Design)	17
4.2 Thiết kế chi tiết.....	19
4.2.1 Thiết kế giao diện	19
4.2.2 Thiết kế lớp nghiệp vụ	19
4.2.3 Thiết kế cơ sở dữ liệu	20
4.3 Xây dựng ứng dụng.....	21
4.3.1 Thư viện và công cụ sử dụng	21
4.3.2 Kết quả đạt được	21
4.3.3 Minh họa các chức năng chính	22
4.4 Kiểm thử và Triển khai.....	22
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	23
5.1 Giải pháp sinh đề thi tự động và xáo trộn đề.....	23
5.1.1 Giới thiệu bài toán.....	23
5.1.2 Giải pháp đề xuất	23
5.1.3 Thuật toán sinh đề thi tự động	23
5.1.4 Cơ chế xáo trộn câu hỏi và đáp án.....	24

5.2 Cơ chế giám sát thi cử (Anti-Cheat Monitoring)	25
5.2.1 Giới thiệu bài toán.....	25
5.2.2 Cấu trúc giám sát Client-Side (Trình duyệt học viên).....	25
5.2.3 Quy trình xử lý đồng bộ và kiểm soát phía Server-Side (Backend) ..	26
5.3 Tích hợp luồng học và thi khép kín.....	26
5.3.1 Mô hình hóa lớp học làm trung tâm	26
5.3.2 Đồng bộ hóa dữ liệu và thông báo bất đồng bộ.....	27
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	28
6.1 Kết luận	28
6.2 Hướng phát triển.....	28
TÀI LIỆU THAM KHẢO.....	31
PHỤ LỤC.....	33
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	33
A.1 Ngành học.....	34
A.2 Đánh dấu (bullet) và đánh số (numering)	34
A.3 Cách thêm bảng	35
A.4 Chèn hình ảnh	35
A.5 Tài liệu tham khảo	36
A.6 Cách viết phương trình và công thức toán học.....	36
A.7 Qui cách đóng quyển.....	36
B. ĐẶC TẢ USE CASE.....	38
B.1 Đặc tả use case “Import câu hỏi từ Excel”	38
B.2 Đặc tả use case “Làm bài thi trực tuyến và tự động lưu”	39

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống JQK Study	7
Hình 2.2	Biểu đồ use case phân rã phân hệ Quản lý người dùng	7
Hình 2.3	Biểu đồ use case phân rã phân hệ Quản lý ngân hàng câu hỏi và đề thi	8
Hình 2.4	Biểu đồ use case phân rã phân hệ Làm bài thi	9
Hình 4.1	Biểu đồ phân tầng và phụ thuộc gói của hệ thống	17
Hình 4.2	Biểu đồ thiết kế chi tiết lớp và mối quan hệ giữa các thành phần	18
Hình A.1	Internet vạn vật	35
Hình A.2	Quy cách đóng quyển đồ án	37
Hình A.3	Quy cách đóng quyển đồ án	37

DANH MỤC BẢNG BIỂU

Bảng 2.1	Bảng so sánh nghiệp vụ hệ thống JQK Study với các giải pháp hiện có	5
Bảng 4.1	Danh sách thư viện và công cụ phát triển hệ thống	21
Bảng A.1	Table to test captions and labels.	35

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này giới thiệu tổng quan về đề tài "Hệ thống thi trắc nghiệm trực tuyến JQK Study". Nội dung tập trung trình bày lý do chọn đề tài, mục tiêu, phạm vi nghiên cứu, cũng như định hướng giải pháp và bố cục của báo cáo.

1.1 Đặt vấn đề

Trong thời đại công nghệ số hóa hiện nay, việc học tập và thi cử trực tuyến đang trở thành một xu hướng tất yếu. Tại các trường đại học, trung tâm đào tạo, cũng như các cơ sở giáo dục khác, nhu cầu có một hệ thống cho phép tổ chức thi trắc nghiệm trực tuyến linh hoạt, bảo mật và hiệu quả là rất lớn. Các phương pháp thi truyền thống thường tiêu tốn nhiều thời gian và nguồn lực trong khâu ra đề, coi thi và chấm điểm. Hơn nữa, với các nền tảng học tập trực tuyến hiện tại, thường xảy ra tình trạng thiếu sự giám sát chặt chẽ, dẫn đến gian lận trong thi cử.

Xuất phát từ thực tế đó, bài toán đặt ra là làm thế nào để xây dựng một hệ thống học tập và thi trắc nghiệm trực tuyến toàn diện, không chỉ giúp giảng viên quản lý ngân hàng câu hỏi, tạo đề thi tự động mà còn cung cấp cơ chế giám sát vi phạm trong quá trình thi. Việc giải quyết bài toán này sẽ mang lại lợi ích to lớn cho cả giảng viên và học viên, giúp tối ưu hóa quá trình kiểm tra đánh giá, đồng thời đảm bảo tính công bằng và minh bạch.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay trên thị trường đã có một số nền tảng hỗ trợ thi trực tuyến, tuy nhiên nhiều hệ thống vẫn còn hạn chế ở khả năng giám sát tự động, tính linh hoạt trong cấu trúc khóa học hoặc khả năng tích hợp đa chức năng (vừa học vừa thi).

Trên cơ sở phân tích đó, đề án này hướng tới mục tiêu xây dựng nền tảng **JQK Study** - hệ thống thi trắc nghiệm trực tuyến hiện đại. Hệ thống được phát triển với các nhóm chức năng chính như sau:

- **Quản lý Bài thi & Câu hỏi:** Cho phép tạo, quản lý ngân hàng câu hỏi, import từ Excel, cấu hình sinh đề thi tự động và xáo trộn đáp án.
- **Giám sát & Thống kê:** Theo dõi hành vi vi phạm (chuyển tab, copy/paste) trong thời gian thực, thống kê điểm số và xuất báo cáo.
- **Quản lý Khóa học & Lớp học:** Hỗ trợ giảng viên tạo khóa học, bài giảng video, quản lý mã tham gia lớp học và gán đề thi cho lớp.

Phạm vi của đề tài tập trung vào việc đáp ứng nghiệp vụ quản lý đào tạo, kiểm tra đánh giá cho ba nhóm người dùng chính là Quản trị viên (Admin), Giảng viên

(Instructor) và Học viên (Student).

1.3 Định hướng giải pháp

Để giải quyết bài toán đặt ra, hệ thống JQK Study áp dụng kiến trúc Microservices để đảm bảo tính mở rộng, khả năng chịu tải tốt và dễ dàng bảo trì trong tương lai.

Về mặt giải pháp nghiệp vụ, đề án xây dựng cơ chế giám sát thi cử chặt chẽ bằng cách ghi nhận các sự kiện vi phạm từ phía người dùng (client-side) và kiểm soát tính hợp lệ của bài nộp trên máy chủ (server-side). Bên cạnh đó, thuật toán sinh đề tự động được thiết kế nhằm phân bổ câu hỏi ngẫu nhiên theo cấu hình (chủ đề, độ khó) mà giảng viên đã định sẵn, đảm bảo các bài thi có chất lượng đồng đều.

Đóng góp chính của đề án là mang lại một giải pháp phần mềm hoàn chỉnh, tích hợp chặt chẽ giữa tính năng học tập (courses) và kiểm tra (exams), kết hợp với quy trình quản lý giám sát nghiêm ngặt, đáp ứng nhu cầu thực tế của các cơ sở giáo dục.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau:

Chương 2 trình bày về khảo sát hiện trạng, phân tích yêu cầu nghiệp vụ và đặc tả các chức năng của hệ thống.

Trong Chương 3, đề án giới thiệu về các công nghệ, nền tảng lý thuyết được sử dụng để xây dựng hệ thống JQK Study.

Chương 4 mô tả thiết kế kiến trúc hệ thống, thiết kế cơ sở dữ liệu và một số kết quả thực nghiệm nổi bật.

Chương 5 trình bày các giải pháp và đóng góp chính của đề tài, tập trung vào nghiệp vụ sinh đề thi tự động và giám sát chống gian lận.

Cuối cùng, Chương 6 tóm tắt lại các kết quả đã đạt được, đồng thời đưa ra những hạn chế và hướng phát triển của đề tài trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này trình bày chi tiết về quá trình khảo sát hiện trạng, phân tích các yêu cầu nghiệp vụ và đặc tả chức năng của hệ thống JQK Study. Từ những phân tích này, đồ án xác định được các tác nhân tham gia và quy trình nghiệp vụ cần thiết để xây dựng phần mềm.

2.1 Khảo sát hiện trạng và các giải pháp liên quan

Để xây dựng một hệ thống học tập và thi cử trực tuyến toàn diện, việc khảo sát các giải pháp hiện hành là vô cùng cần thiết. Quá trình này giúp nhận diện rõ các ưu điểm cần kế thừa và những hạn chế về mặt nghiệp vụ, từ đó định hình các tính năng cốt lõi cho hệ thống đề xuất JQK Study.

2.1.1 Khảo sát các hệ thống giáo dục và thi trực tuyến hiện hành

Hiện nay, trên thị trường giáo dục trực tuyến trong nước và quốc tế có nhiều giải pháp hỗ trợ học tập và tổ chức thi cử. Dưới đây là phân tích chi tiết từ góc độ nghiệp vụ và tính năng của ba hệ thống tiêu biểu:

Moodle (Modular Object-Oriented Dynamic Learning Environment) là hệ thống quản lý học tập (LMS) mã nguồn mở phổ biến nhất hiện nay, được ứng dụng rộng rãi tại các trường đại học và cơ sở giáo dục lớn. Về mặt nghiệp vụ, Moodle cung cấp đầy đủ các hoạt động học tập phong phú như bài giảng slide, diễn đàn thảo luận, bài tập nộp và theo dõi tiến độ. Hệ thống này sở hữu phân hệ quản lý ngân hàng câu hỏi rất mạnh mẽ, cho phép tổ chức câu hỏi theo nhiều danh mục phân cấp và hỗ trợ đa dạng loại câu hỏi như trắc nghiệm, điền từ, so khớp hay tự luận. Đồng thời, Moodle cho phép cấu hình chi tiết các quy tắc thi bao gồm giới hạn thời gian, số lượt làm bài và cách tính điểm thi. Tuy nhiên, giao diện người dùng và quản trị của Moodle khá cồng kềnh, phức tạp với nhiều cấu hình thừa thãi, gây khó khăn cho giảng viên trong việc tạo nhanh đề thi. Quy trình sinh đề thi ngẫu nhiên của hệ thống cũng đòi hỏi phải thiết lập danh mục câu hỏi phức tạp từ trước, và nền tảng này hiện tại vẫn thiếu tính năng giám sát chống gian lận trực tuyến tích hợp sẵn như cơ chế phát hiện, ngăn chặn học viên chuyển tab trình duyệt, sao chép tài liệu hay rời khỏi màn hình làm bài.

Canvas LMS là hệ thống quản lý học tập thương mại dựa trên đám mây, nổi tiếng với thiết kế tập trung vào trải nghiệm người dùng cao cấp tại các cơ sở giáo dục quốc tế. Điểm mạnh của Canvas LMS nằm ở quy trình tổ chức khóa học khoa học, giao diện làm bài thi trực quan giúp học viên dễ dàng theo dõi thời gian cùng trạng thái câu hỏi, và tính năng chấm điểm, phản hồi bài thi rất tốt với công cụ

chấm điểm tập trung thân thiện, dễ sử dụng. Mặc dù vậy, hệ thống này đòi hỏi chi phí sử dụng thương mại rất cao, đồng thời khả năng tùy biến quy trình thi cử và sinh đề tự động theo các tiêu chí kết hợp (như tỷ lệ phân bổ độ khó đi kèm với chủ đề cụ thể) còn nhiều hạn chế và kém linh hoạt. Hơn nữa, các tính năng giám sát thi cử bảo mật chuyên sâu như ngăn chặn sao chép hay cảnh báo rời màn hình không được tích hợp sẵn trong Canvas LMS mà bắt buộc phải liên kết với các dịch vụ giám sát trả phí đắt đỏ của bên thứ ba.

Azota là nền tảng chuyên biệt cho việc giao bài tập và tổ chức thi trực tuyến, được phát triển phục vụ học sinh, giáo viên phổ thông và các trung tâm đào tạo tại Việt Nam. Nghiệp vụ nổi bật của Azota là khả năng tạo đề thi cực kỳ nhanh và tối giản nhờ tính năng tự động nhận diện, tách câu hỏi từ file PDF hoặc Word tải lên. Hệ thống cũng hỗ trợ học viên làm bài thi mượt mà trên cả thiết bị di động, đồng thời tích hợp tính năng giám sát chuyển tab thi cơ bản và báo cáo số lần vi phạm cho giáo viên. Tuy thế, Azota vẫn thiếu phân hệ quản lý học tập (LMS) hoàn chỉnh khi không hỗ trợ tổ chức khóa học với bài giảng video tương tác hoặc theo dõi lộ trình tiến độ học tập có cấu trúc. Bên cạnh đó, nghiệp vụ giám sát chống gian lận còn khá đơn giản, dễ bị học sinh vượt qua bằng các thủ thuật cơ bản hoặc sử dụng các thiết bị phụ trợ bên ngoài.

2.1.2 Bảng so sánh chi tiết các hệ thống

Để có cái nhìn hệ thống, Bảng 2.1 so sánh các hệ thống hiện hành với giải pháp JQK Study dựa trên các tiêu chí tính năng và nghiệp vụ quan trọng.

Tiêu chí	Moodle	Canvas LMS	Azota	JQK Study (Đề xuất)
Quản lý học tập (LMS)	Đầy đủ (Bài giảng, tài liệu, diễn đàn, theo dõi tiến độ)	Đầy đủ (Khóa học, lộ trình học tập, tài liệu)	Hạn chế (Chỉ hỗ trợ giao bài tập và tài liệu đi kèm đề thi)	Đầy đủ (Khóa học, bài giảng video, theo dõi tiến độ bài học)
Quản lý ngân hàng câu hỏi	Rất mạnh (Phân cấp danh mục câu hỏi phức tạp, nhiều định dạng)	Tốt (Quản lý ngân hàng câu hỏi dùng chung cho khóa học)	Cơ bản (Lưu trữ danh sách câu hỏi trích xuất từ đề thi tải lên)	Tốt (Quản lý ngân hàng câu hỏi theo Chủ đề và Phân thi phân cấp)
Sinh đề thi & Trộn đề	Có hỗ trợ nhưng giao diện cấu hình sinh đề ngẫu nhiên phức tạp	Có hỗ trợ chia nhóm câu hỏi nhưng thiếu linh hoạt theo nhiều chiều	Trộn câu hỏi và đáp án nhanh từ đề gốc PDF/Word tải lên	Tự động sinh đề thông minh theo tỷ lệ độ khó và chủ đề định sẵn
Giám sát chống gian lận	Không có sẵn cơ chế chặn chuyển tab hay khóa trình duyệt thi	Yêu cầu cài thêm các tiện ích giám sát có phí từ bên thứ ba	Có phát hiện chuyển tab thi nhưng cơ chế giám sát còn đơn giản	Tích hợp sẵn (Ghi nhận hành vi chuyển tab, sao chép, rời màn hình thi)
Làm bài & Chấm điểm	Nhiều cấu hình nhưng giao diện thi cũ, chấm tự động câu trắc nghiệm	Giao diện thi trực quan hiện đại, chấm tự luận tốt	Đơn giản, chấm tự động trắc nghiệm, chấm tự luận qua ảnh chụp	Hiện đại, tối giản cho học viên; dashboard thống kê điểm và vi phạm cho giảng viên
Quản lý lớp học & Tham gia	Gán học viên vào khóa học bằng tài khoản hệ thống hoặc mã ghi danh	Đồng bộ tài khoản trường học hoặc tham gia qua liên kết khóa học	Quản lý danh sách lớp đơn giản, học sinh vào thi qua SĐT/mã đề	Học viên tham gia lớp học chủ động qua mã code (Class Code), gán đề theo lớp

Bảng 2.1: Bảng so sánh nghiệp vụ hệ thống JQK Study với các giải pháp hiện có

2.1.3 Nhu cầu thực tế và định hướng giải quyết của JQK Study

Từ kết quả khảo sát thực tế và bảng so sánh trên, có thể rút ra một số nhận định nghiệp vụ quan trọng:

- Các hệ thống lớn như Moodle hay Canvas tuy mạnh về tính năng quản lý học tập nhưng lại gặp khó khăn về khả năng phục vụ thi cử quy mô lớn (dễ nghẽn mạng khi học viên cùng nộp bài), đồng thời quy trình giám sát bảo mật phức tạp hoặc tồn kém.
- Các ứng dụng như Azota tuy tiện lợi cho việc tổ chức thi nhanh nhưng lại thiếu hụt các tính năng học tập cốt lõi để học viên tự theo dõi bài giảng video và tiến trình học tập của mình.
- Sự tách biệt giữa học (e-learning) và thi (testing) ở nhiều giải pháp hiện tại

làm giảm trải nghiệm học tập liền mạch của học viên và tăng tải trọng quản lý của giảng viên.

Chính vì vậy, nền tảng **JQK Study** được thiết kế nhằm kết hợp đồng bộ hai phân hệ học và thi:

1. **Đồng bộ hóa Học & Thi:** Cho phép học viên vừa học bài giảng video vừa thực hiện làm bài kiểm tra đánh giá trên cùng một nền tảng duy nhất, giúp kết quả thi và tiến độ học tập được cập nhật đồng bộ.
2. **Quy trình giám sát tích hợp sẵn:** Xây dựng trực tiếp các cơ chế tự động ghi nhận hành vi chuyển tab, sao chép nội dung và rời khỏi phòng thi của học viên ngay tại trình duyệt, lưu trữ nhật ký vi phạm trực quan cho giảng viên mà không yêu cầu cài đặt phần mềm bên thứ ba.
3. **Sinh đề tự động thông minh:** Tự động chọn ngẫu nhiên các câu hỏi từ ngân hàng dựa trên cấu hình tỷ lệ độ khó và chủ đề giảng viên đã định sẵn, giúp cá nhân hóa đề thi và đảm bảo tính khách quan.
4. **Đáp ứng quy mô thi đồng thời lớn:** Đảm bảo lưu trữ nháp bài làm tự động (auto-save) định kỳ từ trình duyệt của học viên lên máy chủ, ngăn chặn tối đa sự cố mất bài làm hoặc nghẽn hệ thống khi có lượng lớn học viên nộp bài thi cùng lúc.

2.2 Tổng quan chức năng

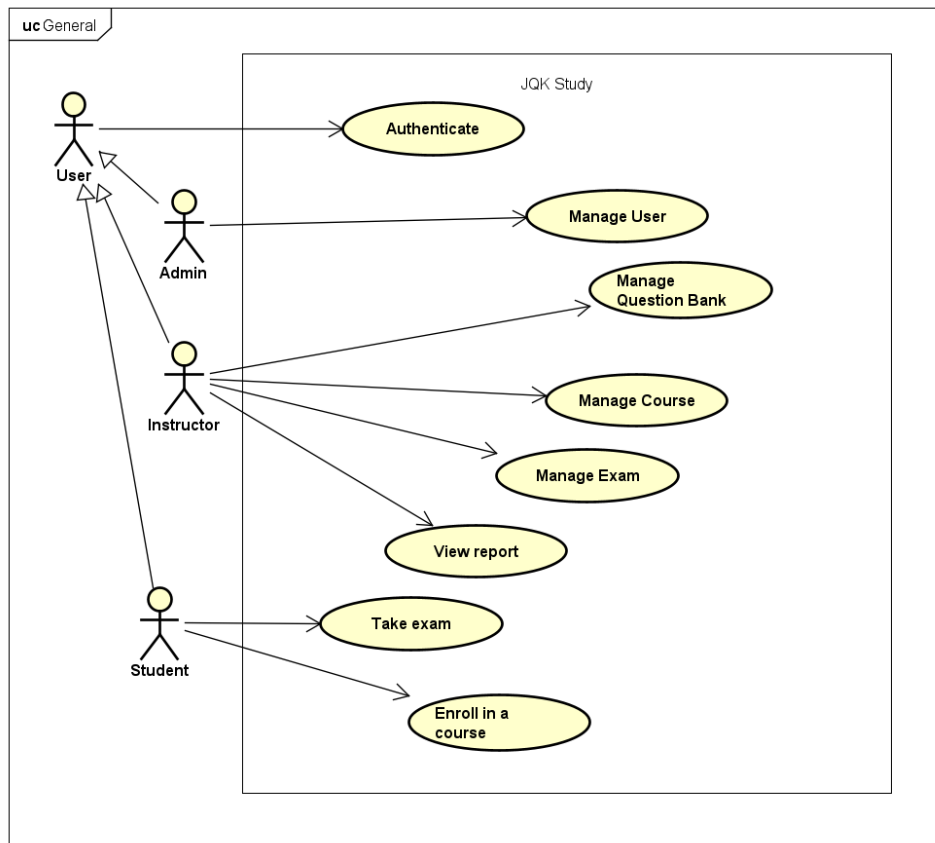
Hệ thống JQK Study được thiết kế phục vụ ba nhóm người dùng (tác nhân) chính: Admin (Quản trị viên), Instructor (Giảng viên) và Student (Học viên).

2.2.1 Biểu đồ use case tổng quát

Các tác nhân tham gia vào hệ thống và vai trò của họ bao gồm:

- **Admin:** Có toàn quyền quản lý hệ thống, quản lý tài khoản người dùng, xem thống kê và có quyền xóa các khóa học không hợp lệ.
- **Instructor:** Giảng viên chịu trách nhiệm về nội dung. Họ có thể tạo khóa học, tạo ngân hàng câu hỏi, sinh đề thi, thiết lập cấu hình bài thi và quản lý lớp học.
- **Student:** Học viên có thể đăng ký khóa học, tham gia lớp học qua mã code, học các bài giảng video và thực hiện bài thi trắc nghiệm.

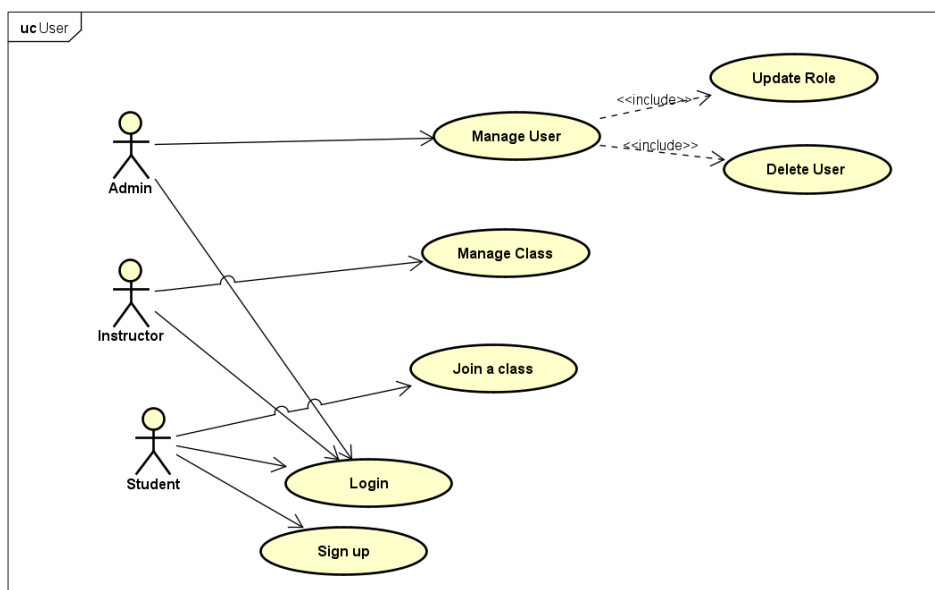
Biểu đồ use case tổng quát mô tả các chức năng chính của từng tác nhân trong hệ thống JQK Study được thể hiện ở Hình 2.1.



Hình 2.1: Biểu đồ use case tổng quát của hệ thống JQK Study

2.2.2 Biểu đồ use case phân rã Quản lý người dùng

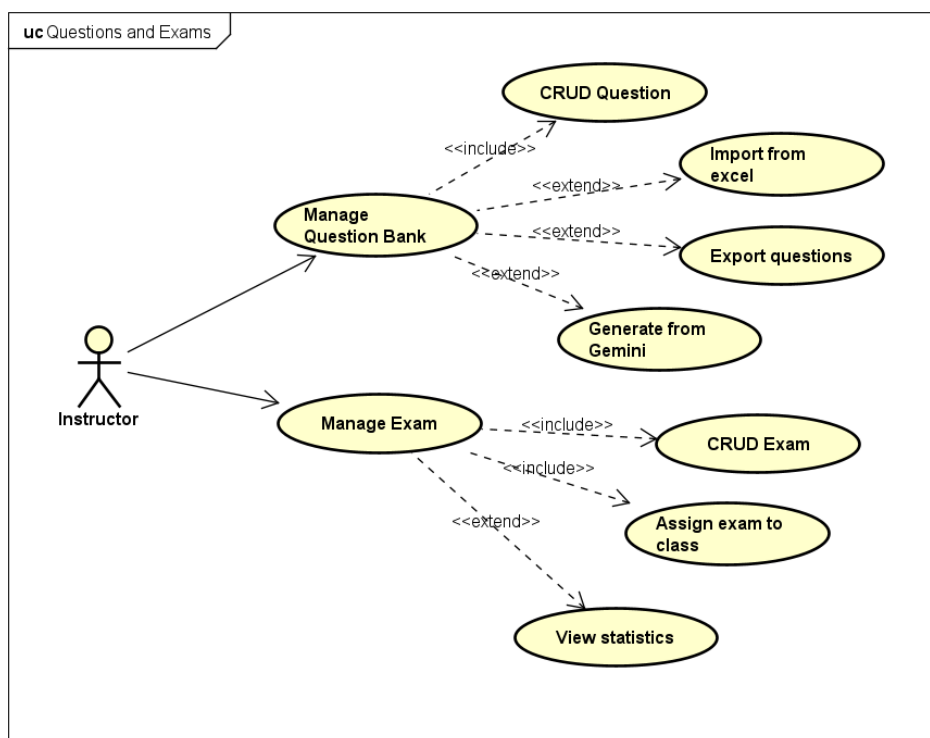
Biểu đồ use case phân rã phân hệ Quản lý người dùng chi tiết hóa các hoạt động đăng ký, đăng nhập, quản lý thông tin tài khoản và quản lý lớp học trong hệ thống JQK Study, được thể hiện chi tiết ở Hình 2.2.



Hình 2.2: Biểu đồ use case phân rã phân hệ Quản lý người dùng

2.2.3 Biểu đồ use case phân rã Quản lý ngân hàng câu hỏi và đề thi

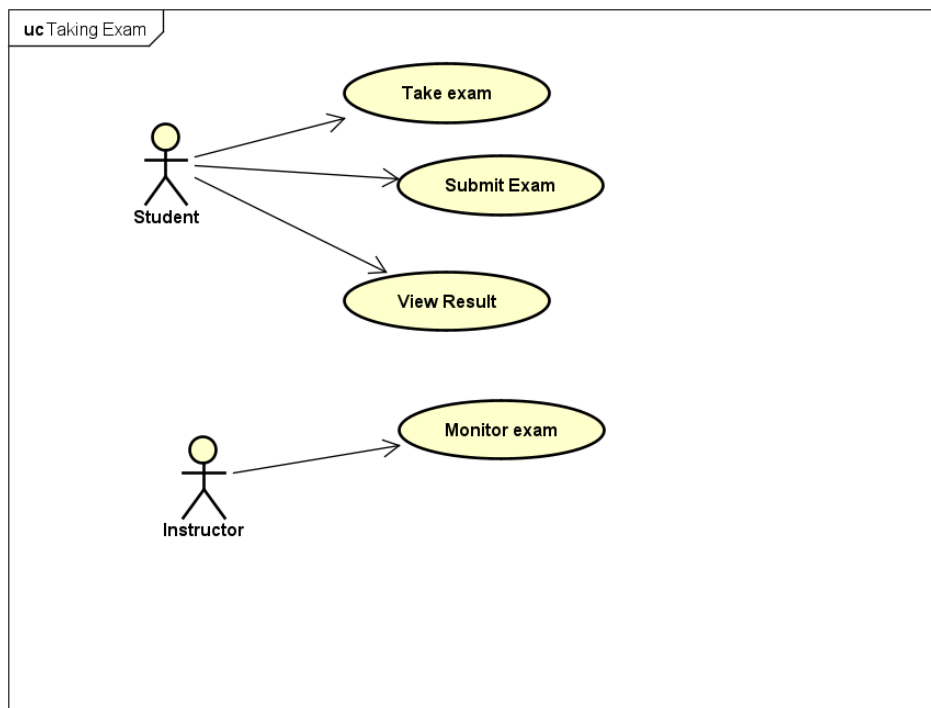
Biểu đồ use case phân rã phân hệ Quản lý ngân hàng câu hỏi và đề thi mô tả chi tiết các chức năng như quản lý chủ đề, thêm/sửa/xóa câu hỏi, import câu hỏi từ Excel và thiết lập các tiêu chí đề thi để sinh đề tự động trong hệ thống JQK Study, được thể hiện chi tiết ở Hình 2.3.



Hình 2.3: Biểu đồ use case phân rã phân hệ Quản lý ngân hàng câu hỏi và đề thi

2.2.4 Biểu đồ use case phân rã Làm bài thi

Biểu đồ use case phân rã phân hệ Làm bài thi mô tả chi tiết các hoạt động của Học viên (Student) khi tham gia phòng thi, trả lời các câu hỏi trắc nghiệm, hệ thống tự động ghi nhận đáp án (auto-save) và ghi nhận hành vi vi phạm (blur tab, copy/paste) trong suốt quá trình làm bài, được thể hiện chi tiết ở Hình 2.4.



Hình 2.4: Biểu đồ use case phân rõ phân hệ Làm bài thi

2.2.5 Quy trình nghiệp vụ

Hai quy trình nghiệp vụ quan trọng nhất của hệ thống là quy trình **Tổ chức thi** và quy trình **Làm bài thi**.

1. Quy trình Tổ chức thi (dành cho Giảng viên):

- Giảng viên tạo Chủ đề (Topic) và Phần (Section) cho ngân hàng câu hỏi.
- Giảng viên thêm câu hỏi thủ công hoặc Import từ file Excel.
- Giảng viên cấu hình Đề thi (thời gian làm bài, số lượt thi, mật khẩu, thuật toán sinh tự động).
- Giảng viên gán đề thi cho một Lớp học cụ thể và phát hành đề thi.

2. Quy trình Làm bài thi & Giám sát (dành cho Học viên):

- Học viên tham gia lớp học và chọn bài thi. Hệ thống kiểm tra điều kiện (mật khẩu, số lượt thi còn lại).
- Khi học viên bắt đầu làm bài, đồng hồ tính giờ (server-side) được kích hoạt.
- Trong quá trình thi, nếu học viên chuyển tab hoặc rời khỏi màn hình, hệ thống sẽ ghi nhận "Vi phạm" và gửi cảnh báo về máy chủ.
- Hết giờ hoặc khi học viên nộp bài, hệ thống tự động chấm điểm, lưu lịch sử vi phạm và hiển thị kết quả.

2.3 Đặc tả chức năng

2.3.1 Đặc tả Use case Sinh đề thi tự động

Tiền điều kiện: Giảng viên đã đăng nhập và có sẵn câu hỏi trong ngân hàng.

Luồng sự kiện chính:

1. Giảng viên chọn chức năng "Tạo đề thi tự động".
2. Hệ thống yêu cầu nhập các tham số: Chủ đề, số lượng câu hỏi, mức độ khó, thời gian thi.
3. Giảng viên xác nhận yêu cầu.
4. Hệ thống chạy thuật toán chọn ngẫu nhiên các câu hỏi thỏa mãn điều kiện và tạo thành đề thi mới.
5. Hệ thống thông báo tạo thành công.

Hậu điều kiện: Đề thi mới được lưu vào cơ sở dữ liệu ở trạng thái Nháp.

2.3.2 Đặc tả Use case Giám sát và Nộp bài thi

Tiền điều kiện: Học viên đang trong phiên làm bài thi.

Luồng sự kiện chính:

1. Học viên chọn đáp án cho các câu hỏi, hệ thống tự động lưu nháp (auto-save).
2. Trình duyệt phát hiện sự kiện 'blur' (chuyển tab), tự động gửi log vi phạm về server.
3. Khi học viên bấm "Nộp bài", hệ thống tính toán điểm số dựa trên đáp án đúng.
4. Hệ thống lưu trạng thái hoàn thành và danh sách các vi phạm trong quá trình thi.

Hậu điều kiện: Điểm số được cập nhật, học viên không thể tiếp tục làm bài thi này.

2.4 Yêu cầu phi chức năng

Để đảm bảo hệ thống JQK Study vận hành ổn định, an toàn và có khả năng phục vụ kỳ thi đồng thời quy mô lớn với hàng ngàn học viên, các yêu cầu phi chức năng về mặt kỹ thuật công nghệ được đặc tả chi tiết như sau:

• Hiệu năng và khả năng mở rộng (Performance & Scalability):

- Hệ thống phải được thiết kế theo kiến trúc vi dịch vụ (Microservices), phân tách rõ ràng các miền nghiệp vụ (User, Course, Exam, Notification) và triển khai bằng ngôn ngữ lập trình Go để đạt tốc độ xử lý nhanh, tối ưu hóa mức độ chiếm dụng tài nguyên hệ thống.

- Có khả năng tự động mở rộng theo chiều ngang (scale-out) thông qua việc đóng gói các dịch vụ thành các container độc lập và điều phối bằng nền tảng Kubernetes để nhanh chóng nâng số lượng máy chủ xử lý khi tải CPU tăng đột biến trong giờ cao điểm làm bài thi.
- Giao tiếp nội bộ giữa các microservices cần sử dụng giao thức gRPC để giảm thiểu độ trễ truyền tin và băng thông mạng so với REST API truyền thống.
- **Tính sẵn sàng và độ tin cậy (Availability & Reliability):**
 - Hệ thống phải duy trì trạng thái hoạt động liên tục (High Availability) với cơ chế cân bằng tải ở API Gateway.
 - Sử dụng bộ nhớ đệm Redis để cache thông tin cấu hình đề thi và thông tin tài khoản, giúp giảm tải truy vấn trực tiếp vào cơ sở dữ liệu chính PostgreSQL.
 - Tích hợp hệ thống hàng đợi thông điệp Apache Kafka để truyền tin nhắn bất đồng bộ đối với các tác vụ nặng (như ghi nhận nhật ký vi phạm thời gian thực, gửi email thông báo kết quả thi), đảm bảo không xảy ra tình trạng mất mát dữ liệu hoặc nghẽn hàng đợi khi sinh viên nộp bài thi dồn dập.
- **Bảo mật và an toàn dữ liệu (Security & Data Integrity):**
 - Xác thực và phân quyền truy cập thông qua cơ chế mã thông báo bảo mật JWT (JSON Web Token), ngăn chặn các truy cập API trái phép.
 - Sử dụng cơ chế kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) để phân quyền chặt chẽ các tài nguyên nghiệp vụ tương ứng cho Admin, Instructor và Student.
 - Toàn bộ mật khẩu người dùng phải được băm một chiều bằng thuật toán mã hóa bcrypt trước khi lưu trữ vào cơ sở dữ liệu PostgreSQL.
 - Dữ liệu bài thi, đáp án làm bài nháp của thí sinh cần được tự động lưu trữ định kỳ lên máy chủ (mỗi 30 giây) để phòng ngừa sự cố mất kết nối mạng hoặc lỗi phần cứng từ phía thiết bị của thí sinh.

Tóm lại, Chương 2 đã đặc tả chi tiết các yêu cầu và chức năng nghiệp vụ thiết yếu của hệ thống, làm nền tảng cho việc thiết kế kiến trúc và lựa chọn công nghệ ở các chương tiếp theo.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày các cơ sở lý thuyết và công nghệ cốt lõi được sử dụng để xây dựng hệ thống JQK Study. Với từng công nghệ được lựa chọn, đề án sẽ phân tích lý do sử dụng, vai trò của nó trong việc giải quyết các yêu cầu nghiệp vụ ở Chương 2, đồng thời so sánh với các giải pháp thay thế để làm rõ tính khoa học của sự lựa chọn.

3.1 Kiến trúc Microservices

Kiến trúc Microservices (vi dịch vụ) là một hướng tiếp cận phát triển phần mềm trong đó ứng dụng được chia nhỏ thành một tập hợp các dịch vụ có kích thước nhỏ, chạy độc lập và giao tiếp với nhau qua mạng thông qua các giao thức nhẹ như HTTP/REST hoặc gRPC.

Giải quyết yêu cầu: Hệ thống thi trắc nghiệm thường có lượng truy cập không đồng đều. Lượng truy cập vào các chức năng học tập thông thường khá thấp, nhưng khi có kỳ thi diễn ra, lượng truy cập vào dịch vụ làm bài thi và ghi nhận vi phạm tăng đột biến. Việc áp dụng Microservices cho phép co giãn (scale-out) riêng biệt dịch vụ thi (*Exam Service*) mà không cần phải nhân bản toàn bộ hệ thống, giúp tối ưu hóa tài nguyên phần cứng.

So sánh lựa chọn thay thế:

- *Kiến trúc Monolithic (Đơn khối):* Dễ xây dựng ở giai đoạn đầu, tuy nhiên khó mở rộng, mã nguồn ngày càng phình to dẫn đến khó bảo trì, và khi một phần nhỏ gặp sự cố (như rò rỉ bộ nhớ ở module thi) có thể làm sập toàn bộ hệ thống. Do đó, đề án quyết định chọn Microservices để đảm bảo tính mở rộng bền vững.

3.2 Ngôn ngữ lập trình Go (Golang)

Go là ngôn ngữ biên dịch mã nguồn mở được phát triển bởi Google, nổi bật với hiệu năng thực thi cao gần tương đương C/C++, cấu trúc cú pháp tối giản và hỗ trợ lập trình song song xuất sắc thông qua khái niệm *Goroutines* và *Channels*.

Giải quyết yêu cầu: Trong thời gian diễn ra bài thi, backend phải xử lý đồng thời hàng ngàn yêu cầu ghi nhận đáp án nhấp (auto-save) và các sự kiện vi phạm gửi về liên tục từ client. Go cung cấp cơ chế non-blocking I/O mạnh mẽ, cho phép chạy hàng chục ngàn luồng nhẹ (*Goroutines*) đồng thời với mức tiêu hao bộ nhớ rất thấp (chỉ khoảng vài KB mỗi *Goroutine*), đáp ứng hoàn hảo yêu cầu hiệu năng cao và thời gian phản hồi nhanh của hệ thống.

So sánh lựa chọn thay thế:

- *Java / C#*: Hiệu năng rất tốt nhưng thời gian khởi động chậm và tiêu tốn nhiều tài nguyên bộ nhớ RAM (thường cần cấu hình RAM tối thiểu lớn cho mỗi service).
- *Node.js (JavaScript)*: Single-threaded nên xử lý các tác vụ tính toán nặng (như xáo trộn đề thi hoặc tính điểm số lượng lớn) có thể gây nghẽn event loop.

Do đó, Go là lựa chọn tối ưu về mặt cân bằng giữa hiệu năng và tài nguyên hệ thống.

3.3 Framework Next.js

Next.js là một framework JavaScript được xây dựng trên nền tảng thư viện React, hỗ trợ các chế độ render linh hoạt bao gồm Server-Side Rendering (SSR), Static Site Generation (SSG) và Client-Side Rendering (CSR).

Giải quyết yêu cầu: Next.js giúp tối ưu hiệu năng hiển thị phía client nhờ cơ chế tải trang nhanh, tự động chia nhỏ code (code splitting). Việc sử dụng App Router thể hệ mới giúp quản lý trạng thái chuyển trang mượt mà, đồng thời hỗ trợ SEO tốt cho các trang khóa học công cộng.

So sánh lựa chọn thay thế:

- *React (Single Page Application - SPA) với Vite*: Tải toàn bộ bundle JS về trình duyệt ở lần đầu truy cập, khiến thời gian tải trang ban đầu (FCP) lâu hơn và khó tối ưu hóa SEO.

3.4 Giao thức truyền thông gRPC và Protocol Buffers

gRPC là một framework gọi hàm từ xa (RPC) hiệu năng cao, mã nguồn mở do Google phát triển, hoạt động trên giao thức truyền tải HTTP/2. gRPC sử dụng Protocol Buffers làm ngôn ngữ định nghĩa giao diện (IDL) và định dạng tuần tự hóa dữ liệu dạng nhị phân siêu nén.

Giải quyết yêu cầu: Trong kiến trúc Microservices, giao tiếp giữa API Gateway và các dịch vụ nội bộ (User, Exam, Course) xảy ra liên tục. Sử dụng gRPC giúp giảm đáng kể kích thước gói tin truyền tải trên mạng và tăng tốc độ xử lý tuần tự hóa dữ liệu so với JSON truyền thống, đảm bảo độ trễ truyền thông liên dịch vụ gần như bằng không.

So sánh lựa chọn thay thế:

- *REST API (JSON over HTTP/1.1)*: Gói tin dạng text có kích thước lớn, giao thức HTTP/1.1 bị giới hạn bởi vấn đề Head-of-Line blocking và không hỗ trợ kết nối ghép kênh (multiplexing) mạnh mẽ như HTTP/2 của gRPC.

3.5 Hệ quản trị cơ sở dữ liệu PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở mạnh mẽ, tuân thủ nghiêm ngặt các tính chất ACID (Atomicity, Consistency, Isolation, Durability) và hỗ trợ rất tốt kiểu dữ liệu JSONB phi cấu trúc.

Giải quyết yêu cầu: Dữ liệu thi cử đòi hỏi độ chính xác tuyệt đối (không được mất điểm, không được sai lệch câu trả lời). Cơ chế transaction của PostgreSQL bảo đảm an toàn dữ liệu. Đồng thời, cấu hình sinh đề thi tự động dưới dạng linh hoạt (dynamic config) được lưu trữ tối ưu dưới định dạng JSONB trong PostgreSQL mà vẫn đảm bảo tốc độ truy vấn cao.

So sánh lựa chọn thay thế:

- *MongoDB (NoSQL)*: Rất linh hoạt nhưng không hỗ trợ tốt các mối quan hệ ràng buộc phức tạp giữa các thực thể như User - Class - Exam - Submission, và các giao dịch nhiều bảng khó đảm bảo tính nhất quán tuyệt đối.

3.6 Hệ thống lưu trữ đệm Redis

Redis là một kho lưu trữ cấu trúc dữ liệu trong bộ nhớ (In-memory), mã nguồn mở, được sử dụng làm cơ sở dữ liệu tạm thời, bộ nhớ đệm (cache) và môi giới tin nhắn.

Giải quyết yêu cầu: Redis lưu trữ các phiên đăng nhập (JWT token), thông tin cấu hình kỳ thi đang diễn ra, và lưu trữ trạng thái làm bài tạm thời của học sinh. Việc này giúp giảm tải đến hơn 70% truy vấn đọc trực tiếp vào cơ sở dữ liệu chính PostgreSQL trong suốt thời gian thi.

3.7 Hệ thống hàng đợi thông điệp Apache Kafka

Apache Kafka là một nền tảng truyền trực tuyến sự kiện phân tán, có khả năng xử lý hàng triệu sự kiện mỗi giây với độ trễ cực thấp và độ tin cậy cao nhờ cơ chế ghi log tuần tự và phân mảnh (partitioning).

Giải quyết yêu cầu: Khi học viên nộp bài thi, ngoài việc tính điểm và lưu cơ sở dữ liệu, hệ thống cần thực hiện nhiều tác vụ phụ như gửi email kết quả, cập nhật tiến độ lớp học, và cập nhật bảng xếp hạng. Thay vì thực hiện đồng bộ gây chậm trễ cho học viên, Exam Service đẩy sự kiện 'ExamSubmittedEvent' vào Kafka. Notification Service sẽ tiêu thụ (consume) sự kiện này để gửi thông báo bất đồng bộ, giúp cô lập sự cố và đảm bảo hệ thống không bị quá tải.

3.8 Công nghệ Container hóa và Điều phối (Docker & Kubernetes)

Docker giúp đóng gói mã nguồn và tất cả các thư viện phụ thuộc của từng service thành một container image duy nhất, đảm bảo tính nhất quán giữa các môi trường

lập trình và triển khai thực tế. Kubernetes (K8s) đảm nhận vai trò tự động hóa việc triển khai, co giãn quy mô và quản lý các ứng dụng container hóa này.

Giải quyết yêu cầu: Đảm bảo hệ thống đạt tính sẵn sàng cao (High Availability). Khi số lượng sinh viên truy cập làm bài tăng vọt, Kubernetes sẽ tự động nhân bản (scale-up) số lượng pod của Exam Service thông qua cơ chế Horizontal Pod Autoscaler (HPA), và tự động phân phối tải đều giữa các pod để tránh tắc nghẽn cục bộ.

Tóm lại, sự kết hợp đồng bộ giữa các công nghệ trên tạo nên một hạ tầng vững chắc cho JQK Study, đáp ứng toàn diện cả yêu cầu chức năng lẫn phi chức năng được đề ra.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày chi tiết về mặt kỹ thuật của hệ thống JQK Study, bao gồm thiết kế kiến trúc phần mềm, thiết kế chi tiết cơ sở dữ liệu và các lớp nghiệp vụ, cùng kết quả xây dựng ứng dụng, phương pháp kiểm thử và mô hình triển khai thực tế.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống JQK Study được phát triển dựa trên ****kiến trúc Microservices**** (kiến trúc vi dịch vụ), chia nhỏ hệ thống thành các khối dịch vụ độc lập chịu trách nhiệm cho từng miền nghiệp vụ riêng biệt. Việc này cho phép mỗi thành phần hoạt động, lưu trữ dữ liệu độc lập và giao tiếp với nhau qua mạng thông qua các giao thức gọi hàm từ xa tốc độ cao gRPC hoặc thông qua hệ thống truyền tin nhắn bất đồng bộ Kafka.

Cụ thể, hệ thống được chia làm hai tầng chính:

- **Tầng Client (Frontend):** Được xây dựng bằng Next.js (chạy ở cổng '4001'), giao tiếp với Backend thông qua các truy vấn REST API gửi tới API Gateway.
- **Tầng Services (Backend):** Bao gồm API Gateway đóng vai trò điểm điều phối trung tâm và các vi dịch vụ nghiệp vụ nội bộ (chạy ở các cổng gRPC riêng biệt).

4.1.2 Thiết kế tổng quan

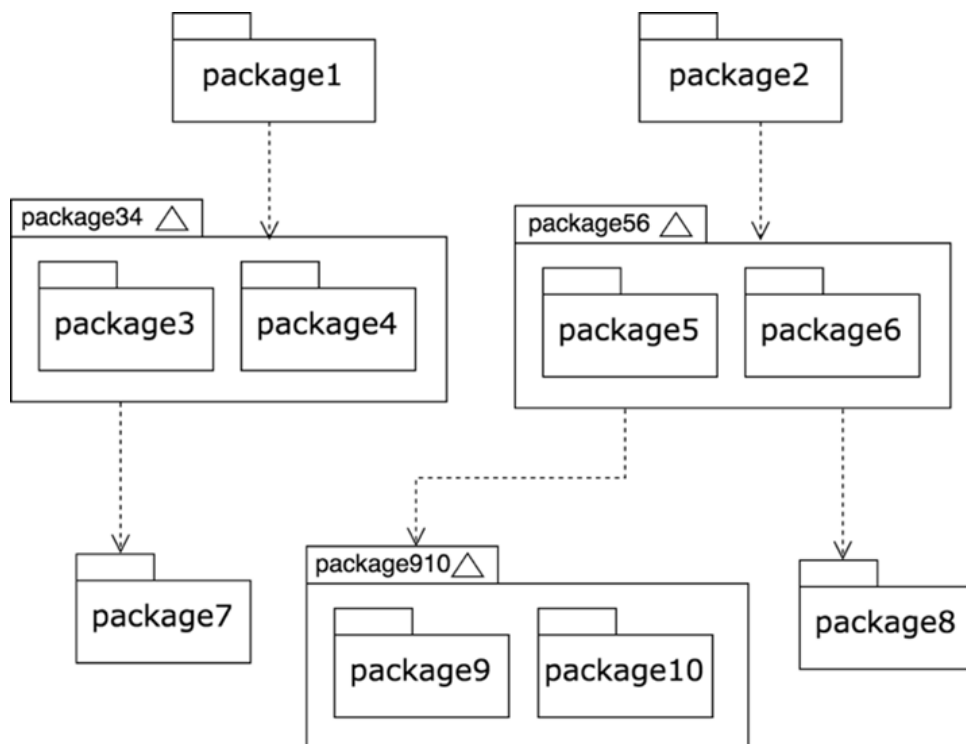
Mô hình kiến trúc tổng quan của JQK Study bao gồm các dịch vụ sau:

- **API Gateway (Gin Web Framework - Port 8081):** Nhận các yêu cầu HTTP từ frontend, thực hiện phân quyền (authorization) qua JWT, kiểm tra CORS, định tuyến (routing) yêu cầu và chuyển đổi chúng thành các cuộc gọi gRPC tới các dịch vụ phía sau.
- **User Service (Port 50051):** Quản lý thông tin người dùng, mật khẩu (mã hóa bcrypt), phân quyền vai trò (Admin, Instructor, Student), quản lý lớp học (Class) và thành viên lớp học.
- **Exam Service (Port 50052):** Dịch vụ lõi quan trọng nhất, quản lý ngân hàng câu hỏi, chủ đề, phần thi, sinh đề thi tự động, xử lý quá trình học viên làm bài thi, lưu đáp án nháp, chấm điểm tự động và ghi nhận nhật ký vi phạm (blur tab, copy/paste).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

- **Course Service (Port 50053):** Quản lý các khóa học, chương học, bài giảng video/tài liệu, đăng ký khóa học của học viên và theo dõi tiến độ hoàn thành bài học.
- **Notification Service (Port 50054):** Hoạt động ở chế độ bất đồng bộ, đăng ký lắng nghe các sự kiện từ hàng đợi Apache Kafka để tự động gửi thông báo qua email hoặc hệ thống cho người dùng khi có sự kiện đăng ký thành công, nộp bài thi hoặc gán lớp học mới.

Sự phụ thuộc giữa các dịch vụ tuân thủ nguyên tắc lỏng lẻo (loose coupling): các dịch vụ nghiệp vụ không trực tiếp gọi cơ sở dữ liệu của nhau mà chỉ tương tác qua gRPC client được định nghĩa trước bằng các tệp giao ước gRPC Protocol Buffers.



Hình 4.1: Biểu đồ phân tầng và phụ thuộc gói của hệ thống

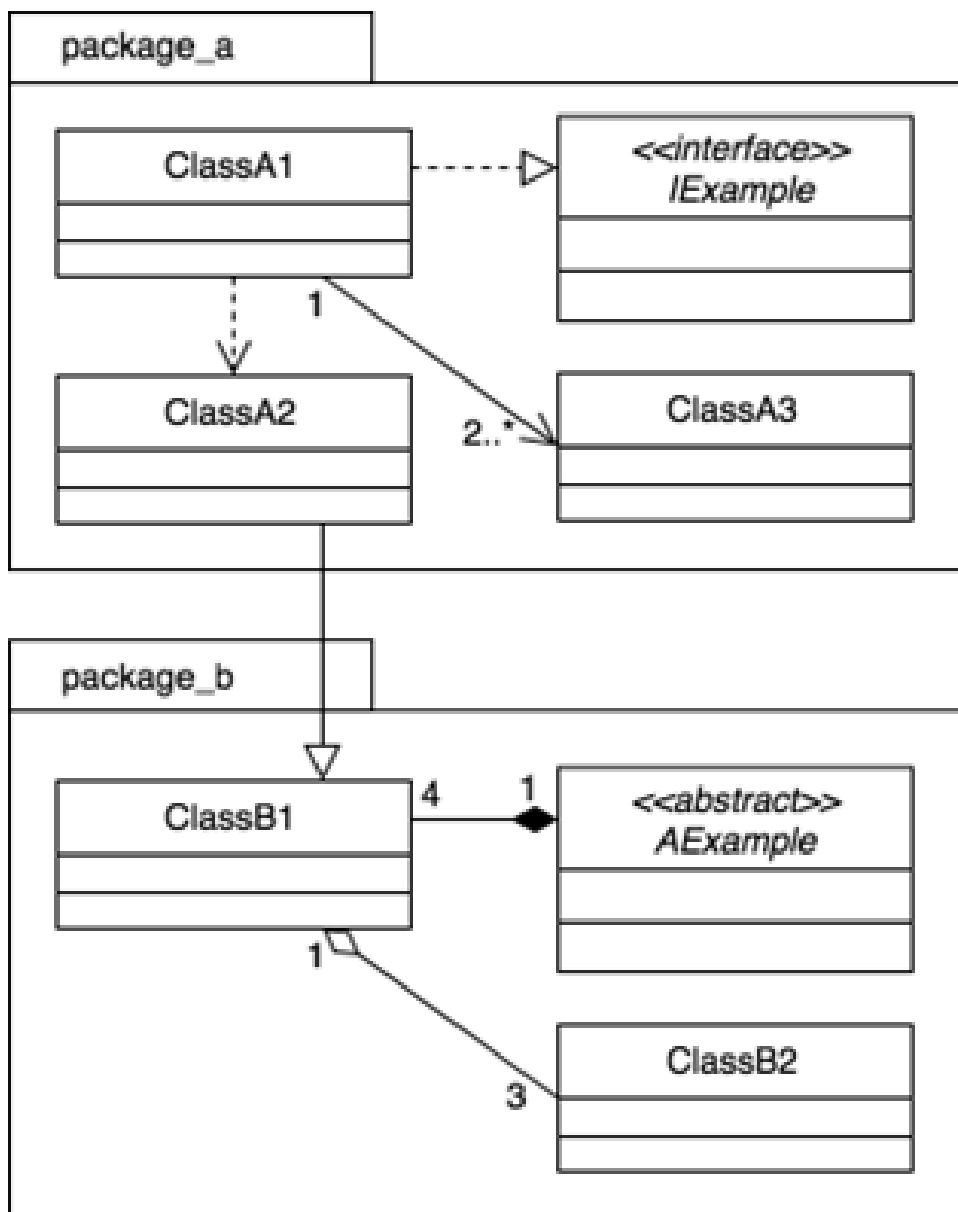
4.1.3 Thiết kế chi tiết gói (Package Design)

Mỗi vi dịch vụ backend trong Go được tổ chức cấu trúc thư mục chuẩn nhằm phân tách rõ ràng các tầng trách nhiệm:

- **Tầng Cmd** (`'cmd/'`): Điểm khởi đầu của ứng dụng, chịu trách nhiệm đọc tệp cấu hình môi trường (`.env`), thiết lập kết nối cơ sở dữ liệu, khởi tạo máy chủ gRPC và lắng nghe các cổng.
- **Tầng Domain** (`'internal/domain/'`): Chứa định nghĩa các mô hình dữ liệu (structs) và các interface cho tầng Service và Repository. Tầng này độc lập

hoàn toàn, không phụ thuộc vào bất kỳ thư viện ngoài nào (Domain-Driven Design).

- **Tầng Service ('internal/service/')**: Nơi chứa logic nghiệp vụ chính (Business Logic), thực hiện kiểm tra điều kiện, xử lý luồng công việc và tương tác với tầng Repository.
- **Tầng Database/Repository ('internal/databases/')**: Thực hiện các câu lệnh truy vấn dữ liệu chi tiết vào cơ sở dữ liệu PostgreSQL hoặc cache Redis sử dụng thư viện GORM.
- **Tầng Infrastructure ('internal/infrastructure/')**: Triển khai gRPC server handler để map dữ liệu gRPC request thành domain model, đồng thời chứa Kafka consumer/producer.



Hình 4.2: Biểu đồ thiết kế chi tiết lớp và mối quan hệ giữa các thành phần

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Giao diện JQK Study được phát triển theo triết lý hiện đại, tương thích tốt với nhiều kích thước màn hình (Responsive Design) từ thiết bị di động đến máy tính để bàn nhờ TailwindCSS. Các quy chuẩn thiết kế giao diện bao gồm:

- ****Hệ màu:**** Sử dụng hệ màu tối (dark mode) làm chủ đạo với các tông màu xám lạnh kết hợp màu xanh ngọc (cyan) tạo nét cao cấp.
- ****Màn hình làm bài thi:**** Được thiết kế tối giản, tập trung tối đa vào nội dung câu hỏi. Có khu vực cố định hiển thị thời gian đếm ngược trực quan và danh sách trạng thái các câu hỏi đã làm/chưa làm.
- ****Giao diện Giảng viên:**** Dashboard trực quan sử dụng thư viện biểu đồ Recharts hiển thị thống kê điểm số, phân phối phổ điểm và danh sách cảnh báo vi phạm chi tiết của từng sinh viên.

4.2.2 Thiết kế lớp nghiệp vụ

Dựa trên kiến trúc mã nguồn nghiệp vụ của các dịch vụ, các interface chính được thiết kế để điều phối hoạt động bao gồm:

1. Phân hệ User Service:

- ‘UserService’: Định nghĩa các phương thức nghiệp vụ đăng ký (‘Register’), đăng nhập (‘Login’), lấy thông tin tài khoản (‘GetProfile’), tạo lớp học (‘CreateClass’), và quản lý thành viên lớp học.
- ‘UserRepository’: Cung cấp giao diện tương tác dữ liệu như lưu người dùng mới (‘CreateUser’), tìm kiếm tài khoản theo email (‘GetUserByEmail’), tạo lớp học (‘CreateClass’), và gán học viên vào lớp.

2. Phân hệ Course Service:

- ‘CourseService’: Thực hiện nghiệp vụ tạo khóa học, quản lý chương học, bài giảng video, ghi nhận tiến độ xem bài giảng và đăng ký khóa học.
- ‘CourseRepository’: Thực hiện lưu trữ thông tin khóa học (‘CreateCourse’), bài học (‘CreateLesson’), lưu tiến trình học tập (‘CreateEnrollment’, ‘UpdateProgress’).

3. Phân hệ Exam Service:

- ‘ExamService’: Xử lý nghiệp vụ tạo đề thi thủ công hoặc tự động (‘CreateExam’, ‘GenerateExam’), lưu trữ câu trả lời của thí sinh theo thời gian thực (‘SubmitAnswer’), chấm điểm tự động (‘SubmitExam’), ghi nhận vi phạm

gian lận và kết xuất kết quả thi.

- ‘ExamRepository’: Quản lý các thao tác ghi dữ liệu đề thi (‘CreateExam’), ngân hàng câu hỏi (‘CreateQuestion’), tạo bài làm mới (‘CreateSubmission’), lưu kết quả làm bài của thí sinh (‘SaveUserAnswer’), và ghi lại vi phạm (‘LogViolation’).

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu quan hệ PostgreSQL với các bảng được phân tách độc lập theo từng vi dịch vụ để đảm bảo tính cô lập dữ liệu.

1. Các bảng dữ liệu của User Service:

- `roles`: Lưu trữ vai trò người dùng (`id`, `name`).
- `users`: Thông tin người dùng (`id`, `full_name`, `email`, `password`, `role_id`, `created_at`, `updated_at`).
- `classes`: Lưu thông tin lớp học (`id`, `name`, `code`, `description`, `teacher_id`, `is_active`).
- `class_members`: Liên kết nhiều-nhiều giữa lớp học và học viên (`class_id`, `user_id`, `role`, `status`, `joined_at`).

2. Các bảng dữ liệu của Course Service:

- `courses`: Lưu thông tin khóa học (`id`, `title`, `description`, `instructor_id`, `price`, `is_published`).
- `course_sections`: Phân chia chương học (`id`, `course_id`, `title`, `order_index`).
- `lessons`: Chi tiết bài học (`id`, `section_id`, `title`, `type_id`, `content_url`, `duration_seconds`, `order_index`).
- `enrollments`: Học viên tham gia khóa học (`user_id`, `course_id`, `enrolled_at`).
- `lesson_progresses`: Theo dõi tiến độ xem bài học (`user_id`, `lesson_id`, `completed_at`).

3. Các bảng dữ liệu của Exam Service (Dịch vụ thi):

- `topics` và `exam_sections`: Quản lý danh mục ngân hàng câu hỏi.
- `questions`: Lưu trữ nội dung câu hỏi, loại câu hỏi (một đáp án/nhiều đáp án), độ khó (dễ, trung bình, khó), giải thích đáp án.
- `choices`: Danh sách các câu trả lời lựa chọn của câu hỏi (`id`, `question_id`,

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

`content, is_correct)`.

- `exams`: Thông tin kỳ thi (`id`, `title`, `duration_minutes`, `start_time`, `end_time`, `max_attempts`, `password`, `is_dynamic`, `dynamic_config`, `status`). Cột `dynamic_config` lưu cấu hình sinh đề dạng JSONB bao gồm tỷ lệ phân bổ độ khó và số lượng câu hỏi cần lấy theo từng chủ đề.
- `exam_questions`: Liên kết danh sách câu hỏi cố định trong đề thi.
- `exam_submissions`: Lưu thông tin bài nộp của học viên (`id`, `exam_id`, `user_id`, `score`, `started_at`, `submitted_at`).
- `user_answers`: Lưu câu trả lời chi tiết của thí sinh đối với từng câu hỏi trong lượt làm bài (`submission_id`, `question_id`, `chosen_choice_id`, `is_correct`, `awarded_points`).
- `exam_violations`: Lưu trữ nhật ký vi phạm (`id`, `exam_id`, `user_id`, `violation_type`, `violation_time`) nhằm lưu bằng chứng khi thí sinh rời màn hình hoặc thực hiện sao chép trong phòng thi.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Các công cụ phát triển phần mềm được sử dụng trong đề án JQK Study được tổng hợp chi tiết trong Bảng 4.1.

Mục đích	Công cụ/Thư viện	Phiên bản/Địa chỉ URL
Ngôn ngữ lập trình Backend	Go (Golang)	v1.25.3 / https://go.dev
Framework giao tiếp API	gRPC	v1.62.0 / https://grpc.io
Bộ thư viện ORM Go	GORM	v1.31.0 / https://gorm.io
Cơ sở dữ liệu chính	PostgreSQL	v16.0 / https://www.postgresql.org
Hệ thống lưu đệm	Redis	v7.0 / https://redis.io
Hệ thống hàng đợi sự kiện	Apache Kafka	v3.5.0 / https://kafka.apache.org
Framework phát triển Frontend	Next.js	v16.0.1 / https://nextjs.org
Đóng gói ứng dụng	Docker	v24.0 / https://www.docker.com
Hệ thống điều phối ứng dụng	Kubernetes	v1.28.2 / https://kubernetes.io
Công cụ phát triển hot-reload	Tilt	v0.33 / https://tilt.dev

Bảng 4.1: Danh sách thư viện và công cụ phát triển hệ thống

4.3.2 Kết quả đạt được

Hệ thống JQK Study đã được xây dựng và đóng gói hoàn chỉnh thành 5 Docker Container độc lập: ‘api-gateway’, ‘user-service’, ‘exam-service’, ‘course-service’, và ‘notification-service’. Tổng số dòng code Go phía backend khoảng 12,000 dòng và Next.js frontend khoảng 8,500 dòng TypeScript. Hệ thống hoạt động trơn tru

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

trên cụm Kubernetes cục bộ được điều khiển bởi Tilt, tự động quản lý kết nối cơ sở dữ liệu và phục vụ giao diện tại cổng 4001.

4.3.3 Minh họa các chức năng chính

Các chức năng chính bao gồm: 1. ****Làm bài thi trực tuyến:**** Màn hình làm bài được khóa tương tác bên ngoài, đồng hồ tự động đếm ngược và cập nhật đáp án về server mỗi 30 giây (auto-save). 2. ****Dashboard giám sát vi phạm:**** Giảng viên xem được danh sách sinh viên vi phạm kèm số lần chuyển tab cụ thể và thời gian vi phạm chính xác. 3. ****Sinh đề thi tự động:**** Giảng viên chọn chủ đề, nhập tỷ lệ độ khó và hệ thống tự động bốc đề ngẫu nhiên từ ngân hàng câu hỏi.

4.4 Kiểm thử và Triển khai

Hệ thống được kiểm thử thông qua các phương pháp kiểm thử đơn vị (Unit Test) cho các hàm xử lý tính toán điểm thi, xáo trộn câu hỏi và kiểm thử tích hợp (Integration Test) cho luồng nghiệp vụ từ khi bắt đầu thi, tự động lưu câu trả lời, gửi cảnh báo vi phạm cho đến khi nộp bài thi thành công.

Về triển khai, hệ thống JQK Study được cấu hình triển khai trên nền tảng Kubernetes. Trong môi trường kiểm thử phát triển, Tilt được cấu hình để tự động hóa quá trình biên dịch mã nguồn, build lại docker image và deploy nóng lên cụm Kubernetes cục bộ mỗi khi lập trình viên lưu mã nguồn. Khi đưa vào vận hành thực tế, hệ thống có thể được dễ dàng phân phối lên các dịch vụ đám mây (AWS EKS hoặc Google GKE) với khả năng tự động co giãn số lượng Pod dựa trên mức độ sử dụng CPU thông qua cấu hình Horizontal Pod Autoscaler.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày chi tiết các giải pháp nghiệp vụ cốt lõi và những đóng góp học thuật nổi bật của đề tài. Quá trình thiết kế và xây dựng hệ thống JQK Study tập trung giải quyết triệt để hai bài toán thực tế đầy thách thức trong giáo dục trực tuyến: (1) tự động hóa quy trình sinh đề thi trắc nghiệm thông minh, khách quan và (2) xây dựng cơ chế giám sát thi cử đa lớp nhằm tối thiểu hóa các hành vi gian lận trên môi trường web.

5.1 Giải pháp sinh đề thi tự động và xáo trộn đề

5.1.1 Giới thiệu bài toán

Trong các hệ thống quản lý đào tạo thông thường, việc ra đề thi thường được thực hiện thủ công: giảng viên tự chọn từng câu hỏi từ ngân hàng hoặc sao chép từ một tài liệu có sẵn. Quy trình này bộc lộ nhiều hạn chế:

- Tốn nhiều thời gian và công sức của giảng viên khi muốn tạo ra một bài đánh giá lớn.
- Khó đảm bảo tính khách quan và phân bố đồng đều về mặt kiến thức cũng như mức độ khó của đề thi.
- Rất khó để tạo ra nhiều mã đề khác nhau có độ khó tương đương nhằm chống sao chép giữa các học viên ngồi cạnh nhau.

5.1.2 Giải pháp đề xuất

Hệ thống JQK Study đề xuất giải pháp sinh đề thi tự động dựa trên cấu hình phân bố động lưu dưới định dạng JSONB trong cơ sở dữ liệu. Giảng viên chỉ cần định nghĩa một cấu trúc khung của đề thi bao gồm: danh sách các chủ đề (Topics), các phần thi (Sections) mục tiêu và tỷ lệ phần trăm các câu hỏi theo các cấp độ nhận thức (Dễ, Trung bình, Khó).

Khi nhận được yêu cầu thi, hệ thống sẽ thực hiện truy vấn và tính toán tự động để lựa chọn ra tập hợp các câu hỏi thỏa mãn các ràng buộc cấu hình thông qua một thuật toán sinh đề ngẫu nhiên có điều kiện.

5.1.3 Thuật toán sinh đề thi tự động

Quy trình lựa chọn câu hỏi tự động được triển khai chi tiết thông qua giải thuật mô tả trong Thuật toán 1.

Algorithm 1: Thuật toán sinh đề thi tự động ngẫu nhiên theo cấu hình

Input: Cấu hình đề thi C (chứa danh sách chủ đề T_C , số lượng câu hỏi yêu cầu N , tỷ lệ độ khó $\{P_{\text{dễ}}, P_{\text{tb}}, P_{\text{khó}}\}$ sao cho

$P_{\text{dễ}} + P_{\text{tb}} + P_{\text{khó}} = 100\%$); Ngân hàng câu hỏi Q trong PostgreSQL.

Output: Đề thi E gồm danh sách N câu hỏi được chọn lọc hoặc thông báo lỗi.

Khởi tạo danh sách câu hỏi của đề thi: $E \leftarrow \emptyset$;

Tính toán số lượng câu hỏi cần lấy cho mỗi mức độ khó:

$N_{\text{dễ}} \leftarrow \lfloor N \times P_{\text{dễ}} \rfloor, \quad N_{\text{tb}} \leftarrow \lfloor N \times P_{\text{tb}} \rfloor, \quad N_{\text{khó}} \leftarrow N - (N_{\text{dễ}} + N_{\text{tb}})$;

for mỗi mức độ khó $d \in \{\text{Dễ}, \text{Trung bình}, \text{Khó}\}$ **do**

Khởi tạo danh sách câu hỏi ứng viên có độ khó d : $Q_{\text{tmp}} \leftarrow \emptyset$;

for mỗi chủ đề $t \in T_C$ **do**

Lọc các câu hỏi trong ngân hàng:

$Q_{t,d} \leftarrow \{q \in Q \mid q.\text{Topic} = t \wedge q.\text{Difficulty} = d \wedge q.\text{IsActive} = \text{True}\}$;

Thêm các câu hỏi lọc được vào danh sách ứng viên:

$Q_{\text{tmp}} \leftarrow Q_{\text{tmp}} \cup Q_{t,d}$;

end

if $|Q_{\text{tmp}}| < N_d$ **then**

return Thông báo lỗi: "Không đủ câu hỏi trong ngân hàng đáp ứng độ khó d cho các chủ đề đã chọn." ;

end

Xáo trộn ngẫu nhiên danh sách ứng viên Q_{tmp} bằng giải thuật

Fisher-Yates (sử dụng hàm sinh số giả ngẫu nhiên có Seed thay đổi) ;

Lấy N_d câu hỏi đầu tiên từ danh sách Q_{tmp} đã xáo trộn và đưa vào E :

$E \leftarrow E \cup Q_{\text{tmp}}[0 \dots N_d - 1]$;

end

Xáo trộn lại toàn bộ danh sách câu hỏi trong E để phân bố ngẫu nhiên các câu hỏi dễ, trung bình, khó xen kẽ nhau ;

return E ;

5.1.4 Cơ chế xáo trộn câu hỏi và đáp án

Để nâng cao hơn nữa khả năng chống sao chép bài thi, JQK Study áp dụng cơ chế xáo trộn hai lớp (Double Shuffling) khi học viên bắt đầu một phiên làm bài thi:

1. **Lớp 1 - Xáo trộn thứ tự câu hỏi:** Mỗi thí sinh khi mở đề thi sẽ nhận được một danh sách câu hỏi có thứ tự hiển thị hoàn toàn khác nhau. Quá trình này được thực hiện bằng cách ánh xạ danh sách câu hỏi gốc của đề thi thông qua một mảng chỉ mục ngẫu nhiên được sinh riêng cho từng lượt nộp bài (submission).

2. **Lớp 2 - Xáo trộn thứ tự đáp án lựa chọn:** Đối với mỗi câu hỏi trắc nghiệm, các lựa chọn trả lời (Choices) cũng được xáo trộn thứ tự hiển thị trước khi render ra trình duyệt của học viên. Hệ thống sử dụng một thuật toán sinh hoán vị ngẫu nhiên trên tập hợp các câu trả lời của câu hỏi đó.

Nhờ cơ chế này, ngay cả khi hai học viên ngồi cạnh nhau nhận cùng một đề thi, câu hỏi hiển thị tại một vị trí bất kỳ sẽ khác nhau và các phương án A, B, C, D của cùng câu hỏi đó cũng được tráo đổi hoàn toàn vị trí.

5.2 Cơ chế giám sát thi cử (Anti-Cheat Monitoring)

5.2.1 Giới thiệu bài toán

Trong các kỳ thi trực tuyến không tập trung hoặc tự do, việc đảm bảo tính trung thực của thí sinh là vô cùng khó khăn do họ có thể dễ dàng chuyển sang các tab khác trên trình duyệt để tra cứu tài liệu, sao chép nội dung từ các nguồn bên ngoài, hoặc mở các ứng dụng chat để trao đổi bài giải.

5.2.2 Cấu trúc giám sát Client-Side (Trình duyệt học viên)

Ở phía giao diện làm bài thi (Frontend Next.js), hệ thống thiết lập một trình giám sát chạy ngầm sử dụng các API tiêu chuẩn của HTML5 để bắt các sự kiện tương tác của người dùng. Cách thức thực hiện chi tiết như sau:

- **Phát hiện chuyển tab hoặc thu nhỏ cửa sổ:** Lắng nghe sự kiện `visibilitychange` của Page Visibility API trên đối tượng `document`. Khi thuộc tính `document.visibilityState` chuyển sang trạng thái `'hidden'`, điều đó chứng tỏ thí sinh đã rời khỏi tab thi hoặc thu nhỏ trình duyệt để mở ứng dụng khác.
- **Phát hiện mất tiêu điểm màn hình thi:** Đăng ký sự kiện `blur` trên đối tượng `window`. Khi thí sinh nhấp chuột ra ngoài phạm vi cửa sổ trình duyệt làm bài (ví dụ click vào thanh Taskbar hoặc ứng dụng chạy song song), sự kiện này lập tức bị kích hoạt.
- **Chặn thao tác Sao chép và Dán (Copy/Paste):** Đăng ký sự kiện `copy`, `cut` và `paste` trên vùng chứa đề thi và bài làm. Sử dụng phương thức `event.preventDefault()` để vô hiệu hóa hoàn toàn hành vi sao chép câu hỏi ra ngoài hoặc dán nội dung từ clipboard vào các trường nhập liệu.

Mỗi khi phát hiện hành vi vi phạm (blur tab hoặc sao chép), hệ thống thực hiện hai bước xử lý nghiệp vụ song song:

1. Hiển thị một cửa sổ cảnh báo (Warning Modal) chặn giữa màn hình làm bài yêu cầu thí sinh tập trung và thông báo số lần vi phạm còn lại trước khi hệ

thống tự động thu bài.

2. Gọi một API bất đồng bộ gửi log vi phạm về server chứa các thông tin: Loại vi phạm (BLUR, COPY, PASTE), mốc thời gian vi phạm chính xác (timestamp) và ID phiên thi.

5.2.3 Quy trình xử lý đồng bộ và kiểm soát phía Server-Side (Backend)

Phía máy chủ (Exam Service) đóng vai trò là chốt chặn cuối cùng kiểm soát tính minh bạch của kỳ thi thông qua các giải pháp kỹ thuật sau:

- **Đồng bộ đồng hồ đếm ngược phía máy chủ:** Thời gian bắt đầu làm bài (started_at) và thời gian kết thúc (due_at) được ghi nhận và lưu trữ trực tiếp trên cơ sở dữ liệu máy chủ khi học viên bấm nút bắt đầu thi. Thời gian đếm ngược hiển thị trên trình duyệt chỉ có vai trò trực quan hóa cho học viên; mọi quyết định về việc chấp nhận đáp án nộp đều dựa trên thời gian máy chủ lúc nhận request. Nếu máy chủ phát hiện thời điểm gửi đáp án vượt quá due_at, hệ thống sẽ từ chối ghi nhận điểm.
- **Ghi nhận nhật ký vi phạm không thể sửa xóa:** Các bản ghi vi phạm được chèn vào bảng exam_violations với ràng buộc khóa ngoại chặt chẽ. Hệ thống không cung cấp bất kỳ API nào cho phép sửa hoặc xóa dữ liệu này, đảm bảo tính khách quan tối đa khi giảng viên thực hiện hậu kiểm.
- **Cơ chế tự động nộp bài (Auto-Submit):** Khi số lượng bản ghi vi phạm trong một lượt thi vượt quá ngưỡng cho phép của cấu hình đề thi (ví dụ quá 3 lần chuyển tab), Exam Service sẽ tự động chuyển trạng thái của lượt thi (submission) sang "Đã hoàn thành" (Completed), chấm điểm các câu hỏi đã làm và khóa quyền truy cập tiếp tục của học viên đối với đề thi đó.

5.3 Tích hợp luồng học và thi khép kín

5.3.1 Mô hình hóa lớp học làm trung tâm

Để giải quyết sự phân mảnh giữa nền tảng e-learning (học tập) và online-testing (thi cử), JQK Study đề xuất mô hình tích hợp lấy thực thể Lớp học (Class) làm trung tâm:

- Giảng viên tạo một Lớp học và liên kết với một Khóa học chứa các bài giảng video. Lớp học này tự động sinh ra một mã ghi danh duy nhất (Class Code).
- Học viên chỉ cần nhập mã code để tham gia lớp học. Từ thời điểm này, học viên được cấp quyền truy cập toàn bộ tài liệu học tập của khóa học liên kết và đồng thời hiển thị danh sách các bài thi trắc nghiệm được gán riêng cho lớp đó.

- Giảng viên có thể cấu hình điều kiện tiên quyết (Prerequisites), yêu cầu học viên phải hoàn thành tối thiểu một số tỷ lệ phần trăm bài học video nhất định mới được quyền mở đề thi trắc nghiệm.

5.3.2 Đồng bộ hóa dữ liệu và thông báo bất đồng bộ

Nhằm đảm bảo hiệu năng cao cho hệ thống dưới tải trọng thi cử lớn, luồng tích hợp học và thi được vận hành bất đồng bộ:

- Khi học viên nộp bài thi thành công, Exam Service lập tức tính điểm tự động và phát đi một sự kiện `ExamSubmittedEvent` vào hàng đợi tin nhắn Apache Kafka.
- Dịch vụ Notification Service lắng nghe sự kiện này, tự động kết xuất kết quả thi thành một mẫu báo cáo trực quan và gửi email kết quả thi cho học viên, đồng thời gửi cảnh báo đến email của giảng viên nếu học viên có số lần vi phạm vượt ngưỡng quy định.
- Tiến độ học tập và điểm số được cập nhật đồng bộ về cơ sở dữ liệu tập trung, giúp giảng viên có thể xem trực tiếp các biểu đồ phân tích phổ điểm và mức độ chuyên cần của cả lớp học trên Dashboard quản lý.

Tóm lại, những giải pháp nghiệp vụ và đóng góp kỹ thuật nêu trên đã tạo nên một hệ thống JQK Study toàn diện, giải quyết được cả khía cạnh tối ưu hóa công tác chuẩn bị đề thi của giảng viên lẫn việc thắt chặt công tác coi thi, đảm bảo chất lượng giảng dạy và kiểm tra đánh giá trực tuyến.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trải qua quá trình nghiên cứu, phân tích và phát triển, đề án đã hoàn thành mục tiêu xây dựng nền tảng "Hệ thống thi trắc nghiệm trực tuyến JQK Study". So với các nền tảng truyền thống thường phân mảnh giữa học và thi, JQK Study đã tích hợp thành công hai luồng nghiệp vụ này vào chung một hệ thống, mang lại trải nghiệm liền mạch cho người dùng.

Những đóng góp nổi bật của hệ thống bao gồm:

- **Tự động hóa ra đề:** Giúp giảng viên tiết kiệm thời gian đáng kể thông qua thuật toán sinh đề thi ngẫu nhiên dựa trên cấu hình chủ đề và độ khó.
- **Giám sát thi cử hiệu quả:** Xây dựng được cơ chế chống gian lận (Anti-Cheat) theo thời gian thực bằng việc bắt các sự kiện chuyển tab, copy/paste, đảm bảo tính công bằng của kỳ thi.
- **Quản lý khép kín:** Mô hình tổ chức Lớp học - Khóa học - Đề thi giúp quản lý tiến độ học tập và điểm số một cách thống nhất, tập trung.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế nhất định như chưa hỗ trợ đa dạng hơn các loại hình câu hỏi tự luận hay câu hỏi có tính tương tác cao, cơ chế giám sát chưa áp dụng công nghệ nhận diện khuôn mặt qua AI để tăng cường bảo mật.

6.2 Hướng phát triển

Trong tương lai, để hoàn thiện và nâng cao năng lực của hệ thống JQK Study, các hướng phát triển tiếp theo bao gồm:

- **Tích hợp AI trong giám sát (Proctoring):** Ứng dụng trí tuệ nhân tạo để phân tích hình ảnh từ webcam, nhận diện khuôn mặt và phát hiện các hành vi bất thường của người thi.
- **Đa dạng hóa câu hỏi:** Mở rộng hệ thống để hỗ trợ câu hỏi tự luận, bài tập lập trình (chấm code tự động) nhằm phục vụ các ngành học đặc thù.
- **Phân tích dữ liệu học tập (Learning Analytics):** Áp dụng các kỹ thuật khai phá dữ liệu để phân tích kết quả bài thi, từ đó đưa ra các gợi ý cá nhân hóa nhằm cải thiện lộ trình học tập cho từng học viên.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong DATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: `ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z`
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml`
- [8] S. Scott-Hayward, G. O’Callaghan, and S. Sezer, *Software Defined Networks*. IEEE, 2013.
- [9] K. Ashton et al., “That ‘internet of things’ thing,” *RFID journal*, vol. 22, no. 7, pp. 97–114, 2009.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tối ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

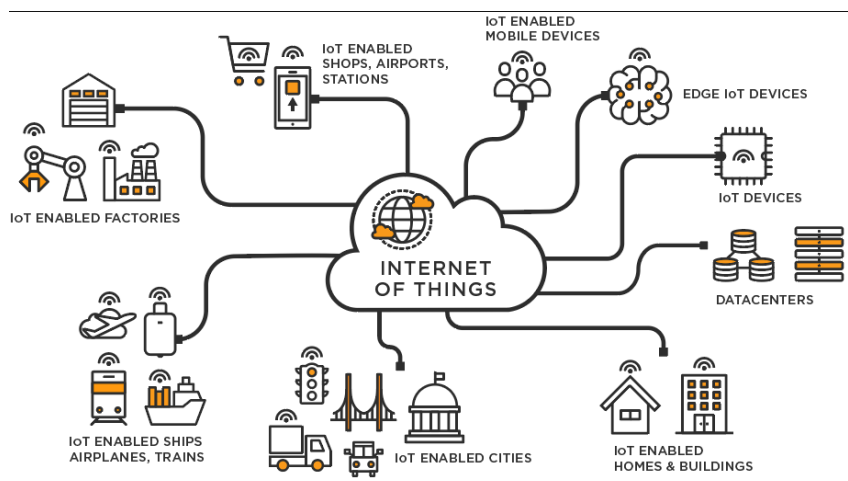
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```



Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [8]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [9]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

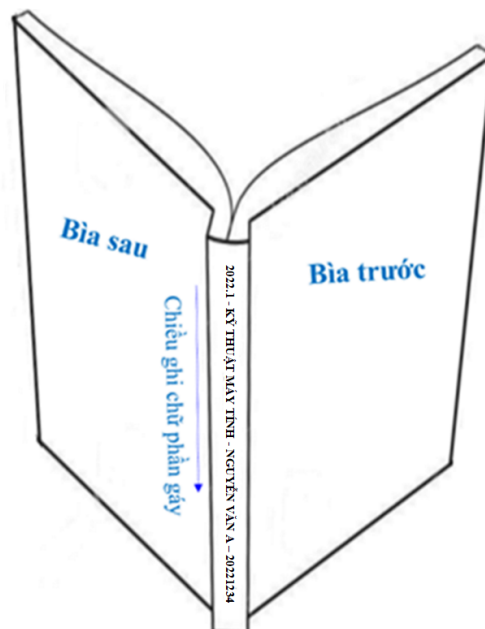
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Quy cách đóng quyển đồ án



Hình A.3: Quy cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Phụ lục này trình bày các đặc tả chi tiết cho một số use case quan trọng khác của hệ thống JQK Study nhằm bổ sung chi tiết cho các phân tích ở Chương 2.

B.1 Đặc tả use case “Import câu hỏi từ Excel”

1. Tác nhân: Giảng viên (Instructor).

2. Mục đích: Giúp giảng viên nhanh chóng đưa hàng trăm câu hỏi vào ngân hàng câu hỏi của hệ thống mà không cần nhập thủ công từng câu một.

3. Tiền điều kiện: Giảng viên đã đăng nhập thành công vào trang quản trị và chuẩn bị sẵn tệp Excel chứa câu hỏi đúng định dạng quy định của hệ thống.

4. Luồng sự kiện chính:

1. Giảng viên truy cập vào giao diện quản lý câu hỏi và chọn mục "Import từ Excel".
2. Hệ thống hiển thị liên kết tải xuống tệp mẫu (Template file) nếu giảng viên chưa có.
3. Giảng viên tải lên tệp Excel chứa danh sách câu hỏi cần Import và chọn chủ đề (Topic) đích.
4. Hệ thống tiếp nhận tệp tin, tiến hành kiểm tra tính hợp lệ của cấu trúc tệp (đúng số cột, đúng định dạng).
5. Hệ thống đọc từng dòng dữ liệu:
 - (a) Phân tích nội dung câu hỏi, độ khó, và giải thích.
 - (b) Đọc danh sách các lựa chọn đáp án tương ứng và cờ đánh dấu đáp án đúng.
 - (c) Lưu thông tin câu hỏi và đáp án vào cơ sở dữ liệu.
6. Hệ thống thông báo kết quả import thành công và số lượng câu hỏi đã được thêm mới.

5. Luồng sự kiện phụ/Ngoại lệ:

- *Tệp tin tải lên sai cấu trúc hoặc trống:* Hệ thống hủy bỏ giao dịch import, thông báo lỗi định dạng tệp và hướng dẫn người dùng tải lại tệp mẫu.
- *Dữ liệu trong dòng bị thiếu trường bắt buộc (như nội dung câu hỏi hoặc không có đáp án đúng):* Hệ thống bỏ qua dòng đó, tiếp tục import các dòng hợp lệ khác và đưa ra thông báo cảnh báo chi tiết các dòng bị lỗi để giảng viên sửa đổi sau.

6. Hậu điều kiện: Các câu hỏi hợp lệ được lưu trữ thành công trong ngân hàng câu hỏi dưới chủ đề đã chọn.

B.2 Đặc tả use case “Làm bài thi trực tuyến và tự động lưu”

1. Tác nhân: Học viên (Student).

2. Mục đích: Học viên thực hiện làm bài thi trắc nghiệm trực tuyến, hệ thống đảm bảo kết quả làm bài của học viên được lưu giữ liên tục để tránh rủi ro mất kết nối.

3. Tiền điều kiện: Học viên đã đăng ký tham gia lớp học có gán đề thi đang mở, đáp ứng đủ điều kiện về số lượt làm bài và mật khẩu bài thi (nếu có).

4. Luồng sự kiện chính:

1. Học viên nhấn chọn "Bắt đầu làm bài thi".
2. Hệ thống ghi nhận mốc thời gian bắt đầu làm bài trên server, khởi tạo lượt thi (Submission) ở trạng thái "Đang làm" (In Progress), và hiển thị danh sách câu hỏi cùng đồng hồ đếm ngược thời gian.
3. Học viên đọc và chọn đáp án cho từng câu hỏi.
4. Định kỳ mỗi 30 giây hoặc ngay khi học viên chọn đáp án, trình duyệt tự động gửi yêu cầu API lưu nháp (auto-save) các câu trả lời hiện tại về máy chủ.
5. Hệ thống cập nhật các lựa chọn đáp án tạm thời vào bảng `user_answers` ứng với Submission ID.
6. Khi hoàn thành, học viên bấm "Nộp bài". Hệ thống tiến hành khóa giao diện làm bài, tính điểm tự động và chuyển trạng thái Submission sang "Đã nộp" (Submitted).

5. Luồng sự kiện phụ/Ngoại lệ:

- *Học viên mất kết nối internet trong khi thi:* Giao diện hiển thị cảnh báo mất kết nối. Sau khi mạng phục hồi, hệ thống tiếp tục tự động lưu nháp dữ liệu mà không làm gián đoạn bài làm của học viên.
- *Thời gian làm bài đếm ngược về 0:* Hệ thống tự động kích hoạt tiến trình nộp bài từ phía máy chủ với các đáp án nháp đã được lưu gần nhất và kết thúc lượt thi của thí sinh.

6. Hậu điều kiện: Kết quả bài thi được lưu trữ an toàn, học sinh xem được điểm số (nếu cài đặt đề thi cho phép).